

Optimus Prime - A VGA Prime Finding Machine

Group 11 - Final Project Report
Dawson Gulasa & Michael See



Table Of Contents

Abstract.....2

I. Introduction and Project Goals2

II. System Overview3

III. Hardware Platform.....5

IV. Design and Implementation.....6

V. Verification and Testing9

VI. Evaluation of the Design10

VII. Reflections and Future Work13

VIII. Team Contributions15

References15

Appendix A. Vivado IP Walkthroughs.....16

Link to GitHub Repository Containing Source Files: <https://github.com/Herring-UGACSEE-4280-S26/Group11/tree/main/Final%20Project>

Link to Downloadable Project: <https://drive.google.com/file/d/1us1Pf--TYopDqmZ6g2jWWlgMPsd5FxaX/view?usp=sharing>

Link to Full Demo: <https://share.icloud.com/photos/0d2bQ99g3ISXTddD8aXxbbevA>

Link to “End of 90 Minutes” Video: <https://share.icloud.com/photos/0968py97sDSILUosE84xacDxQ>

Abstract

This project presents an FPGA-based prime number computation system implemented on the Nexys A7 platform, featuring a VGA-driven user interface and multiple operational modes including range-based generation, time-constrained computation, single-value primality testing, and a verification mode using SD card data. Prime computation is performed using a hardware-optimized trial division algorithm that evaluates divisibility up to the square root of each candidate, with modulus operations implemented via iterative subtraction to ensure synthesizability and predictable timing within a finite state machine architecture. The design emphasizes user experience through an interactive VGA interface with real-time feedback, dynamic visual elements, and mouse-driven navigation, while maintaining a hierarchical Verilog structure separating input, computation, storage, and rendering subsystems. Computed primes are stored in DDR2 memory and later retrieved for large-scale validation in test mode. Functional verification was performed using self-checking testbenches with randomized inputs, edge-case coverage, and forced-failure scenarios. The final system demonstrates correct operation across all modes, reliable memory integration, and a responsive user interface, successfully meeting the project's base, commitment, and prioritization goals.

I. Introduction and Project Goals

This project implements a complete FPGA-based system for prime number computation with an integrated VGA user interface on the Nexys A7 platform. The design combines real-time numerical computation, memory management, and interactive user control within a hierarchical Verilog architecture. In addition to performing prime calculations, the system supports multiple operational modes, large-scale data storage using DDR2 memory, and verification against externally stored reference data. The goal of this project is not only to compute primes correctly, but to do so within a structured hardware design that emphasizes usability, reliability, and clear system organization.

1.1 Problem Statement

Efficiently computing prime numbers in hardware presents several challenges, particularly when combined with real-time user interaction and large data handling. Prime computation requires iterative arithmetic operations and conditional logic, which must be mapped to a finite state machine while maintaining predictable timing and synthesizability. Additionally, integrating this computation with a responsive user interface introduces complexity in coordinating input handling, display updates, and system control across multiple modules. The problem addressed in this project is the design of a unified FPGA system that can

perform accurate prime number computation, manage and verify large datasets, and provide a clear and interactive user experience through a VGA-based interface.

1.2 Required Functional Modes

The system supports four primary operational modes, each designed to meet specific project requirements:

- **Range Mode:** Computes all prime numbers up to a user-defined limit and records the total count and elapsed time.
- **Time Mode:** Computes as many prime numbers as possible within a specified time constraint, reporting the number of primes found.
- **Single Mode:** Determines whether a single user-provided number is prime and reports the result and computation time.
- **Test Mode:** Compares internally computed prime values stored in DDR2 memory against a reference dataset read from an SD card, reporting pass/fail results, indicating missing data, and number of comparisons.

Each mode shares the same underlying computation hardware but differs in control flow, termination conditions, and output behavior.

1.3 Prioritization Goal

The primary prioritization goal for this project was user experience and interface responsiveness. Design decisions were made to ensure that the system is intuitive to operate, visually informative, and responsive to user inputs. This includes real-time display updates, clear presentation of results, and smooth navigation through the interface using mouse control and keypad inputs. While computational efficiency remains important, the system was structured to balance performance with usability, ensuring that users can easily interact with and understand the system's behavior during operation.

1.4 Success Criteria

Success for this project is measured along three axes that map directly to the project's goal hierarchy. The base goal is met when the four functional modes (Range, Time, Single, and Test) all produce correct results on hardware. The commitment goal is met when those results are presented through a polished, mouse- and keypad-driven VGA interface that is responsive enough to feel interactive. The prioritization goal of user experience is met when a user unfamiliar with the design can sit down at the board, navigate the menus, run a mode, and understand the results without external instruction.

II. System Overview

2.1 High-Level System Description

Optimus Prime is a fully integrated FPGA-based system that performs prime number computation while providing a real-time, interactive VGA user interface. The design combines numerical processing, memory management, and user interaction into a cohesive architecture implemented on the Nexys A7 platform. The system is structured hierarchically, separating functionality into distinct subsystems including input control, computation, memory storage, display rendering, and external data verification.

At a high level, the system operates as a data-driven pipeline. User inputs from the mouse, keypad, and pushbuttons are first processed by the input subsystem and converted into structured parameters. These parameters are passed to the prime computation subsystem, which executes a hardware-optimized trial division algorithm to determine primality and generate prime sequences. Computed results are stored in DDR2 memory, which serves both as a framebuffer for VGA output and as large-scale storage for prime data.

The display subsystem continuously renders system state to a 640×480 VGA output using a double-buffered framebuffer architecture to ensure smooth, tear-free visuals. A cursor overlay and sprite-based UI elements provide intuitive navigation and feedback. In parallel, the SD card subsystem enables external validation by streaming reference prime data into the system, allowing the Test Mode controller to compare stored results against known correct values.

The system is designed to operate across multiple clock domains, with explicit synchronization mechanisms ensuring reliable communication between the CPU, VGA, SD card, and DDR2 memory interfaces. This modular and concurrent architecture allows computation, rendering, input handling, and verification to occur simultaneously, resulting in a responsive and robust system that satisfies both functional correctness and user experience goals.

2.2 Top-Level Block Diagram

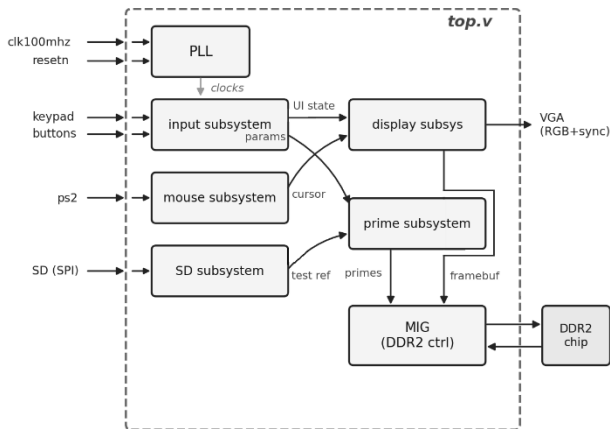


Figure 1: System's Block Diagram

The top-level block diagram of the system is shown in Figure 1. At the center is `top.v`, which acts purely as a wiring layer. It instantiates the major subsystems, declares the inter-module wires, and routes physical board pins to and from those subsystems.

Five subsystem blocks hang off `top.v`. The input subsystem collects all of the user's keypad and pushbutton activity. The mouse subsystem handles the PS/2 mouse, decodes packets, and tracks the cursor. The prime computation subsystem performs the primality work itself and pushes its results into DDR2. The SD subsystem reads the reference prime list from the microSD card at boot. The display subsystem covers everything from the UI screen renderers down to the VGA output drivers.

Two shared resources sit alongside the subsystems. The PLL block generates four working clocks from the 100 MHz board oscillator and provides the system-wide locked signal. The DDR2 controller, instantiated from the MIG IP, runs in its own clock domain and exposes a single user-side port that the framebuffer path and the prime-list path both share through small CDC bridges.

External board pins enter and leave the diagram on its perimeter: the 100 MHz clock and reset on one side, the keypad and pushbutton inputs on another, the mouse PS/2 lines, the microSD pins, the VGA color and sync outputs, and the DDR2 pins. The user's input, the prime engine's output, and the live UI state all converge through `top.v` to produce what appears on the monitor.

2.3 Top-Level Hierarchy

Subsystems are labeled in bold

Input Control / Management

- **input_ctrl.v**
- param_entry.v
- **mouse_subsystem.v**

Prime Engine

- **prime_subsystem.v**
- prime_ddr_bridge.v (wrapper module for 8 sync_ffs)

Output Rendering

- sprite_ctrl.v
- ddr2_fb_ctrl.v
- ui_frame_renderer.v
- vga_output_with_cursor.v

SD-Card Management for Test Mode

- **sd_file_byte_source.v** (SD subsystem)
- sd_byte_sync.v
- sd_prime_parser.v
- sd_prime_feeder.v
- test_mode_ctrl.v

Clock / Pulse Synchronization

- async_fifo.v
- pulse_sync.v
- sync_ff.v (for mouse x, y, and clicks)

2.4 Data Flow Through the System

The system has five data flows that mostly run in parallel. Each one is handled by its own group of modules, and top.v wires them all together.

1. Boot-time: SD card into prime cache

At power-on, the FPGA immediately starts reading data.txt off the microSD card, which contains a list of pre-computed prime numbers, one per line.

- sd_file_byte_source.v talks to the SD card hardware and produces a stream of bytes.
- sd_byte_sync.v safely passes each byte from the SD card's clock domain into the CPU clock domain.
- sd_prime_parser.v watches the byte stream, recognizes ASCII digits, and converts text like "31\n" into the binary number 31.
- sd_prime_cache.v stores all parsed primes in an on-chip array so they can be replayed later during Test Mode.

2. User input: mouse / keypad / button to UI state

Three input devices feed the UI in parallel.

- Mouse: mouse_ps2_ctrl and mouse_packet_decoder read PS/2 packets. mouse_cursor_ctrl.v turns those packets into cursor X/Y coordinates and a left-click signal.
- Keypad: keypad_input_ctrl.v scans the Pmod keypad and decodes which digit (0–9) or letter (B = Backspace, C = Clear) is pressed.
- Buttons: btn_input_ctrl.v debounces the five on-board pushbuttons.

All three converge on input_ctrl.v and param_entry.v. param_entry.v is what tracks which input box is selected and assembles the user's typed digits into the parameter values that drive prime computation.

3. Prime computation: parameters to DDR2 memory

When the user clicks Start, ui_fsm.v (the main UI state machine) sends a go pulse to the compute engine.

- prime_controller.v receives the mode (Range / Time / Single) and the user's parameters, then walks through candidate numbers one at a time.
- Each candidate gets handed to prime_check_core.v, which performs the actual primality test and returns prime or not prime.
- Every prime found is shipped through prime_storage_frontend.v and prime_ddr_bridge.v into off-chip DDR2 memory via ddr2_fb_ctrl.v.

The controller keeps a running count, the largest prime found, and a small history of the most-recent primes for displaying them live.

4. Display: UI state to VGA port

The display pipeline produces a 640×480 image at 60 Hz.

- ui_fsm.v decides which screen the user is on (menu, params, generating, results, test, controls).
- screen_mux.v switches between our six screen renderers (screen_menu.v, screen_params.v, screen_generating.v, screen_results_error_test.v, screen_controls.v).
- ui_frame_renderer.v turns the chosen screen into a stream of pixel words and writes them into a frame buffer in DDR2 memory (double-buffered).
- vga_output_with_cursor.v reads pixels back out of DDR2, overlays the mouse cursor sprite on top, and drives the actual VGA color and sync pins.

5. Test Mode: stored primes vs. SD primes

Test Mode is our verification path that compares our engine's results to static data from the SD card.

- `test_mode_ctrl.v` walks through the primes saved in DDR2 (via `prime_storage_frontend.v`) one at a time.
- In parallel, it pulls primes from the boot-time cache (`sd_prime_cache.v`) one at a time.
- Each pair is compared. If every pair matches all the way to end-of-file, PASS. If any pair differs (or the counts disagree), FAIL. The controller keeps the two mismatched values so they can be shown on screen.

III. Hardware Platform

3.1 FPGA Board Components Used

The design is implemented on a Digilent Nexys A7-100T development board. Of the on-board peripherals provided by the Nexys A7, this project actively wires seven groups of physical resources. The 16 user LEDs and the eight-digit seven-segment display are not used. They were wired during early bring-up for debug visibility but have been removed from the finished product because the on-screen UI now reports the same status information directly to the user.

The first active group is the system clock and reset path. A single 100 MHz oscillator drives the FPGA on package pin E3 through the top-level port `clk100mhz`. The board's CPU RESET pushbutton drives the active-low `resetn` input. These two signals are the only inputs that bypass the user clock domains. Both of them feed directly into the on-chip clock manager that produces the design's working clocks.

The second group is the five pushbuttons: BTNC, BTNU, BTND, BTNL, and BTNR. These five buttons are debounced and edge-detected inside `input_ctrl` and exposed to the UI for soft reset purposes. In the beginning stage of development, they were used for testing UI navigation.

The third group is the Pmod KYPD 4×4 matrix keypad, which plugs directly into the JA Pmod header. Four `kp_col` lines are driven outward by the FPGA in a one-cold scanning pattern. Four `kp_row` lines are sampled back through internal pull-ups. The keypad scanner decodes the resulting row patterns into digit and command keypresses for parameter entry.

The fourth group is the PS/2 mouse path. This path routes through the board's USB-HID bridge, so a normal USB mouse appears to the FPGA as a PS/2 device. Two open-collector lines carry the bidirectional clock and data signals: `ps2_clk` on pin F4 and `ps2_data` on pin B2, both with on-chip pull-ups. The `mouse_subsystem` module initializes the device, decodes the PS/2 byte stream into 3-byte motion packets, and converts those packets into bounded cursor coordinates.

The fifth group is the VGA output. Each color channel uses four bits: `RED[3:0]`, `GRN[3:0]`, and `BLU[3:0]`. Two timing signals, `HSYNC` and `VSYNC`, complete the interface. The design drives 640×480 at 60 Hz pixel timing produced from a dedicated 25 MHz pixel clock.

The sixth group is the on-board microSD slot, used in SPI mode. The slot's eight signals are mapped to FPGA pins as follows: `sd_clk` on B1, `sd_cmd` on C1, `sd_dat0_miso` on C2, `sd_dat1_unused` on E1, `sd_dat2_unused` on F1, `sd_dat3_cs_n` on D2, `sd_reset_n` on E2, and `sd_card_detect` on A1.

The seventh group is the DDR2 SDRAM. The board provides a Micron MT47H64M16HR-25E device wired with a 16-bit physical data bus. This memory is used as Optimus Prime's framebuffer and mass-storage memory. It is driven by a Xilinx Memory Interface Generator IP block, also known as MIG 7-Series.

3.2 Clocking and Clock Domains

All synchronous logic in the project runs from a small set of clocks produced by a single Xilinx PLL instance called `pll`. The PLL takes the 100 MHz board oscillator as its reference and emits four output clocks plus a locked indicator. A board-wide synchronous reset called `sys_resetn_w` is formed by ANDing the external `resetn` button with the PLL locked signal, so no downstream flip-flop comes out of reset until every clock is stable. There are five distinct clock domains in the design.

Name	Waveform	Period (ns)	Frequency (MHz)
clk100mhz	(0.000 5.000)	10.000	100.000
clk_cpu_pll	(0.000 5.000)	10.000	100.000
> clk_mem_pll	(0.000 2.500)	5.000	200.000
clk_sd_pll	(0.000 10.000)	20.000	50.000
clk_vga_pll	(0.000 20.000)	40.000	25.000
clkfbout_pll	(0.000 5.000)	10.000	100.000
sys_clk_pin	(0.000 5.000)	10.000	100.000
clk_cpu_pll_1	(0.000 5.000)	10.000	100.000
> clk_mem_pll_1	(0.000 2.500)	5.000	200.000
clk_sd_pll_1	(0.000 10.000)	20.000	50.000
clk_vga_pll_1	(0.000 20.000)	40.000	25.000
clkfbout_pll_1	(0.000 5.000)	10.000	100.000

Figure 2: Clock Domains Used

The first domain is `clk_cpu` at 100 MHz. This is the design's main system clock and the largest domain by far. It carries the UI screen FSM, the screen multiplexer, the keypad and

pushbutton input controller, the mouse subsystem, the SD-file parser, the prime-list cache, the test-mode controller, and the upstream half of the framebuffer renderer.

The second domain is `clk_vga` at 25 MHz, used exclusively for VGA timing. Everything downstream of the framebuffer-to-VGA crossing runs in this domain, including the horizontal and vertical counters, the pixel pipeline, and the sprite overlay.

The third domain is `clk_sd` at 50 MHz, which clocks the SD-card SPI controller and the file-byte stream produced from it.

The fourth domain is `clk_mem` at 200 MHz, which serves as the reference clock for the DDR2 memory interface. No user logic runs on this clock, but it must be present for MIG to come out of calibration.

The fifth domain is `ui_clk` at roughly 100 MHz, generated inside the MIG IP as the user-side interface clock. All transactions on the MIG user port are synchronous to `ui_clk`, including read and write commands, the 128-bit user-data bus, and the `init_calib_complete` handshake. Although `ui_clk` is nominally the same rate as `clk_cpu`, the two are produced by different PLLs and are treated as fully asynchronous for CDC purposes.

The UI must read from and write to DDR2 while running its own logic on `clk_cpu`, and the mouse and VGA paths cross between `clk_vga` and `clk_cpu`. To handle these crossings safely, the design contains a small family of explicit clock-domain-crossing modules. Slow-changing signals such as cursor coordinates, button states, the `pll_locked` flag, and the `init_calib_complete` flag go through the project's `sync_ff` two-flop synchronizer or through the wider `mouse_cursor_cdc` block. The `sd_byte_sync` module carries the SD-byte stream from `clk_sd` into `clk_cpu`. The `frame_renderer_cdc` module carries pixel-row data from `clk_cpu` into `clk_vga` for display. The `ddr2_fb_cdc` and `prime_ddr_bridge` modules carry framebuffer and prime-list traffic between `clk_cpu` and `ui_clk` over small async FIFOs, so neither side has to stall on the other.

3.3 Memory Architecture

The design uses three places to store data: a large external memory chip on the board, smaller memory blocks built into the FPGA, and a handful of small registers scattered through the logic. Each piece of data is placed on the smallest and fastest storage that can hold it.

The largest store is the board's external DDR2 memory, accessed through the Xilinx MIG IP. The design uses it for two things. The first is the screen image. The renderer writes the full picture into DDR2 one pixel at a time, and the VGA output reads it back line by line while it draws to the monitor. The second is the long list of primes. In the full build, this

list can hold every prime number up to one hundred million, which works out to a little over 5.7 million entries. A smaller debug build, selected by a single `localparam` in `top.v`, cuts the list down to 4096 entries so the design can be tested faster.

The middle tier is on-chip Block RAM. These blocks live inside the FPGA, are small compared to DDR2, and can be read every clock cycle. The design uses them for data that is looked at often. That includes a cache of the primes loaded from the SD card at boot, the font used to draw all the on-screen text, the small bitmaps for the cursor and the corner logos, and the short FIFOs that buffer data while it crosses between clock domains.

The smallest tier is plain registers and a few-bit memory cells built from logic. These hold whatever the design needs to remember from one cycle to the next: the numbers the user types in for the lower and upper bounds, which screen is currently being shown, the intermediate values flowing through the rendering pipeline, and the small flags that track which subsystem is busy.

IV. Design and Implementation

This section describes the seven subsystems that make up Optimus Prime. Each subsection covers the role of the subsystem, the modules that implement it, the finite state machines that drive it, and the interfaces it exposes to the rest of the design.

4.1 Input Subsystem

Role and Modules

The input subsystem collects everything the user does with the keypad and pushbuttons and turns it into clean events the rest of the design can react to. Three modules do the work. The keypad scanner reads the 4×4 Pmod keypad, the button input controller debounces the five on-board pushbuttons, and `input_ctrl` combines both feeds into one shared bus. A fourth module, `param_entry`, sits on top of `input_ctrl` and handles digit entry into the on-screen number boxes.

The keypad scanner drives one column low at a time and watches the four rows. When a row pulls low, it knows which key was pressed. The result is a key code for a digit, a B for backspace, or a C for clear. The button controller does plain debouncing. It samples each pushbutton, waits for it to hold steady for a few milliseconds, and emits a one-cycle pulse on the press edge.

Parameter Entry FSM

`param_entry.v` is what builds the numbers the user actually wants to compute over. It tracks which input box is currently highlighted, then watches the keypad output and updates that box. A digit shifts the current value left by ten and adds the new digit on the right. Backspace divides by ten. Clear sets

the box back to all zeros. The same module also enforces an upper limit on the number of digits, so the user cannot overflow the value or run off the end of the on-screen field.

4.2 Mouse Subsystem

Role and Modules

The mouse subsystem handles everything from the raw PS/2 lines on the board to the cursor X, Y, and button states the rest of the design uses. It is built from four small modules. `mouse_ps2_ctrl` talks to the PS/2 hardware and produces a stream of bytes. `mouse_packet_decoder` groups those bytes into the standard three-byte PS/2 mouse packet. `mouse_cursor_ctrl` turns each packet into an updated cursor position and button state. `mouse_cursor_cdc` safely carries that position and button state from the system clock domain into the VGA clock domain, which is where the cursor actually gets drawn.

PS/2 Initialization and Packet Decode

When the FPGA powers up, the PS/2 controller waits a short time for the mouse to settle, then sends the byte 0xF4 to enable streaming reports. After that, it just listens. Each PS/2 frame is 11 bits long: one start bit, eight data bits, a parity bit, and a stop bit. The controller checks parity and the stop bit on every frame and only releases a byte if both look correct. The packet decoder then collects three of those bytes at a time. The first byte holds the button states and the sign and overflow bits. The second and third bytes hold the X and Y movement.

Cursor Position and Click Generation

`mouse_cursor_ctrl` keeps a running X and Y position, both 10 bits wide so they can cover the full 640×480 screen with a little headroom. Each new packet adds the signed X delta and subtracts the signed Y delta, since the PS/2 standard treats positive Y as upward. The result is then clamped so the cursor cannot move outside the visible area. Button states are forwarded directly. A simple edge detector on the left button line generates the single-cycle click pulse that the UI uses to act on menu hits.

4.3 UI Control Subsystem

Screens and Navigation

The UI is built around six screens: a main menu, a parameter entry screen, a generating screen, a results screen, a test mode screen, and a controls screen. Each screen is its own small module, and `screen_mux` selects which one is rendered at any given moment. Each screen produces the same kind of output, a stream of pixel words that the framebuffer renderer turns into actual pixels in DDR2.

UI / Navigation FSM

`ui_fsm` is the brain of the UI. It tracks which screen the user is on, listens for menu clicks and Start and Back actions, and decides when to move between screens. Most transitions are

direct. A click on a menu tile jumps straight to that mode's parameter screen, a click on Start begins computation, and a click on Back returns to the main menu. The FSM also coordinates the start and stop pulses that the prime engine and the test controller listen for, so that compute work does not begin until the user has confirmed it.

Cursor, Sprite, and Color Usage

The cursor sprite is a small bitmap stored in an on-chip ROM. It is drawn on top of whatever the active screen produces, so it always appears regardless of which mode is running. The same sprite path is reused for the corner logos, which are simply additional bitmaps drawn at fixed positions. Color use is intentionally limited. Only a handful of colors appear across the UI, which keeps the palette consistent from screen to screen and makes it easy for the user to tell what is interactive and what is not.

4.4 Prime Computation Subsystem

Algorithm Choice

The prime check uses trial division up to the square root of the candidate. For a candidate N, the engine tries every divisor from 2 up to the largest integer whose square is at most N. If any of those divisors divides N evenly, N is not prime. If none do, N is prime. Because hardware division is expensive, the design implements the modulus by repeated subtraction. It keeps subtracting the divisor from a running remainder until the remainder is smaller than the divisor, then checks whether what is left is zero. This approach avoids using a divider IP and gives predictable timing.

Example:

$N = 17$

Check Divisors up to $\sqrt{17} \approx 4$

$17 \% 2 \neq 0$

$17 \% 3 \neq 0$

$17 \% 4 \neq 0$ So, 17 is a prime!

Prime Check Core FSM

The core has three working states. In the first state, it loads the candidate and the next divisor. In the second state, it runs the subtraction loop until the remainder drops below the divisor. In the third state, it checks the result, decides whether to advance the divisor or declare the candidate prime, and returns to the first state. A done flag is raised when the next divisor would exceed the square root, at which point the candidate is reported as prime.

Prime Controller FSM

The controller runs above the core and handles the active mode. In Range Mode it walks candidates from the user's lower bound up to the upper bound. In Time Mode it walks until a timer says the time is up. In Single Mode it simply

hands the user's number to the core once and waits for the answer. As primes come back from the core, the controller increments the running count, updates the largest prime seen, and pushes the value into the prime storage front-end so it ends up in DDR2.

4.5 DDR2 Storage Subsystem

Framebuffer Path

The framebuffer path moves pixels from the renderer to the monitor. `ui_frame_renderer` turns the active screen into a stream of pixel words and writes them through `ddr2_fb_cdc` into DDR2. The CDC block carries the writes from the system clock domain across to the MIG user clock domain over a small async FIFO, so the renderer never has to wait on the memory controller. Reads run on the same idea in reverse. `vga_output_with_cursor` pulls each line out of DDR2 just before it is needed and feeds it to the pixel pipeline.

Prime-List Path

The prime-list path uses a separate set of bridge modules so the framebuffer and the prime engine cannot interfere with each other. Found primes go through `prime_storage_frontend`, which packs them and forwards them to `prime_ddr_bridge`. That bridge moves the writes across into the MIG clock domain, again over an async FIFO. Reads work the same way during Test Mode, when the controller pulls primes back out one at a time.

DDR2 / Memory Control Strategy

DDR2 is split into two regions: a framebuffer region for the screen image and a long region for the prime list. The framebuffer is double-buffered, so the renderer always writes to one buffer while the VGA path reads from the other. After each full frame the two buffers swap. The whole subsystem is gated by MIG's `init_calib_complete` signal. Until calibration finishes, no module is allowed to issue reads or writes, which keeps the bring-up sequence safe and predictable.

4.6 SD Test Subsystem

Boot-Time Cache Load

As soon as the FPGA comes out of reset, the SD subsystem starts streaming `data.txt` off the microSD card. `sd_file_byte_source` talks to the card in SPI mode and produces one byte at a time. `sd_byte_sync` carries each byte from the SD clock domain into the system clock domain. `sd_prime_parser` watches the byte stream, recognizes ASCII digits and newlines, and converts text like `"31\n"` into the binary number 31. Each parsed prime is pushed into `sd_prime_cache`, which holds them in on-chip memory for replay during Test Mode.

Test Mode Controller FSM

When the user starts Test Mode, `test_mode_ctrl` walks both lists in parallel. From DDR2 it pulls the primes the engine generated. From the on-chip cache it pulls the reference primes loaded at boot. On every step it compares the two values. If they match, it advances. If they differ, it stops and remembers both values so they can be displayed. The controller also tracks the running comparison count, which is shown to the user as the test progresses.

End-of-Mode Display Behavior

The end of Test Mode falls into one of three outcomes. If every pair matched all the way through, the screen reports PASS along with the total number of comparisons. If a pair differed, it reports FAIL and shows the first mismatched pair. If the engine produced fewer or more primes than the SD reference, the controller reports a count mismatch instead, which makes it clear that the two data sets did not even line up in length.

4.7 VGA / Cursor Subsystem

Pixel Pipeline

The pixel pipeline runs in three stages. In stage zero, the current X and Y from the VGA timing block are used to fetch the matching pixel word from DDR2. In stage one, that word is latched alongside the cursor and sprite information for the same position. In stage two, the final color is selected and driven onto the VGA color outputs. The three-stage layout keeps each stage short and lets the design comfortably hit the 25 MHz pixel rate.

Sprite Overlay

Sprites are drawn on top of the framebuffer pixel just before the color leaves the chip. The cursor and the corner logos all share the same overlay path. Each sprite is a small one-bit-per-pixel bitmap stored in an on-chip ROM. When the current pixel sits inside a sprite's bounding box, the sprite color replaces the framebuffer color. This keeps the sprite drawing logic completely separate from the renderer and means moving the cursor never requires re-rendering the screen.

Timing Generation

The VGA timing block produces standard 640×480 at 60 Hz pixel timing from a 25 MHz pixel clock. It runs two counters. A horizontal counter resets every line, and a vertical counter increments at the end of each line. The counters generate HSYNC and VSYNC at the right offsets, expose a visible-area flag that gates the color outputs, and provide the X and Y the rest of the pipeline uses to know which pixel is being drawn at any given moment.

V. Verification and Testing

5.1 Verification Philosophy

The verification strategy for this project followed a bottom-up, modular testing approach, where each subsystem was independently validated before full system integration. Each module was paired with a self-checking testbench that automatically evaluated correctness and produced PASS/FAIL summaries.

All testbenches followed a consistent structure:

- Directed edge-case testing (boundary conditions, reset behavior, invalid inputs)
- Randomized testing to increase coverage
- Scoreboard/reference models for correctness comparison
- Automated PASS/FAIL reporting
- Optional forced-failure mode to verify that the testbench itself is not trivially passing

For example, the DDR-backed storage testbench uses:

- A behavioral DDR model
- Randomized latency injection
- A reference memory model for comparison

At the subsystem level, integration testbenches verify:

- Event streams (prime_found signals)
- Storage correctness
- FSM completion behavior
- Interaction between modules (controller, storage, parser, etc.)

This approach ensured correctness at every level before progressing to higher-level integration.

5.2 Testbench Coverage Summary

Tb_prime_check_core:

- Level: Unit
- Primary Coverage: Primality checking correctness
- Result: Pass

Tb_prime_controller:

- Level: Unit
- Primary Coverage: FSM sequencing, event generation
- Result: Pass

Tb_prime_ddr_storage:

- Level: Unit
- Primary Coverage: DDR write/read correctness
- Result: Pass

Tb_sd_prime_parser:

- Level: Unit
- Primary Coverage: ASCII to Integer Parsing
- Result: Pass

Tb_sd_prime_feeder:

- Level: Unit
- Primary Coverage: FIFO + handshake correctness
- Result: Pass

Tb_input_ctrl:

- Level: Unit
- Primary Coverage: Button/Keypad debouncing
- Result: Pass

Tb_mouse_subsystem:

- Level: Unit
- Primary Coverage: PS/2 Mouse Packet Decoding
- Result: Pass

Tb_vga_output_with_cursor:

- Level: Unit
- Primary Coverage: VGA timing + cursor overlay
- Result: Pass

Tb_prime_subsystem:

- Level: Integration
- Primary Coverage: Full compute pipeline
- Result: Pass

Tb_test_mode_ctrl:

- Level: Integration
- Primary Coverage: DDR vs SD comparison FSM
- Result: Pass

Tb_top_smoke:

- Level: System
- Primary Coverage: Top-level integration sanity
- Result: Pass

All testbenches produced zero errors in normal mode and demonstrated correct failure behavior when forced-fail mode was enabled.

5.3 Higher-Level / Subsystem Verification

Higher-level testbenches validated full system behavior:

Prime Subsystem (tb_prime_subsystem)

- Verified SINGLE, RANGE, and TIME modes
- Compared outputs against a software reference model
- Checked storage correctness and event streams

Test Mode Controller (tb_test_mode_ctrl)

- Verified comparison between DDR-stored primes and SD card data
- Confirmed correct PASS/FAIL detection and mismatch reporting

Top-Level Smoke Test (tb_top_smoke)

- Ensured full system integration stability
- Verified UI state transitions remain valid under random inputs

These integration tests confirmed correct interaction between all major subsystems.

5.4 Hardware-Level Validation

In addition to simulation, hardware validation was performed using:

VGA output

- Verified rendering of UI, sprites, & cursor behavior

LED indicators

- Used to monitor FSM states and debug signals

7-segment display

- Displayed key values for debugging input/output

SD card input

- Confirmed correct parsing of decimal prime data

DDR2 storage

- Verified storage and retrieval of computed primes

The VGA-based UI provided immediate visual confirmation of system correctness.

5.5 Boundary-Case Testing

Special attention was given to edge cases:

Prime computation:

- $N = 0, 1$ (non-prime); $N = 2$ (first prime)

Range mode:

- $\text{start} > \text{limit}$; very small ranges

Time mode:

- early termination at time limit

Test mode:

- No stored data condition
- First mismatch detection
- End-of-file marker ('A') handling

Input systems:

- Button bounce conditions; Keypad repeated presses

VGA:

- Cursor clipping at screen edges

These cases were explicitly tested in both directed and randomized scenarios.

5.6 Simulation Results

All Testbenches completed with zero errors in normal operation, and verified correct failure detection using forced-fail modes. Example results are shown in **Appendix B**.

Overall, the verification strategy demonstrates a high level of confidence in system correctness across all modules and integration levels.

VI. Evaluation of the Design

6.1 Goal Completion Review

The system meets every primary functional requirement laid out in the project description. The four operational modes (Range, Time, Single, and Test) all run end to end on hardware and produce correct results. Computed primes are stored in DDR2 and later replayed during Test Mode for comparison against the SD card reference. Each mode freezes its display at completion and includes a clear visual indicator that the run is finished. The required display elements are all present, including the running prime count, the list of the most recent primes, the live elapsed time, and a moving sprite drawn on top of the framebuffer.

The technical requirements are also satisfied. VGA serves as the primary user interface, with double buffering in DDR2 to prevent tearing. The SD card holds the reference prime list for testing. DDR2 holds both the framebuffer and the prime storage. The Pmod KYPD keypad fulfills the PMOD requirement, and a USB mouse adds a second interactive input through the on-board PS/2 bridge. The seven-segment

display and the on-board LEDs are not used in the final product because the on-screen UI already provides the same status feedback. The design is fully hierarchical Verilog, follows the class design rules, and is verified with self-checking testbenches at the unit level and on hardware.

Over-and-Beyond Features

Beyond the minimum requirements, the project includes several features intended to make the system feel like a finished product rather than a checklist demo:

- A six-screen UI (menu, parameter entry, generating, results, test mode, and controls) with consistent layout and color use across all screens.
- Mouse-driven navigation with a sprite cursor and full click handling, layered on top of any active screen.
- A 4×4 matrix keypad for digit entry, with backspace and clear keys, alongside per-screen highlighted input boxes.
- A boot-time SD reader that parses the reference prime file into on-chip memory before the user reaches the menu, so Test Mode starts without a load delay.
- Decorative corner logos and a small visual identity for the system.

These features are not required by the rubric, but they directly support the project's prioritization goal of user experience.

6.2 Prioritization Goal Outcomes

The team chose user experience and interface responsiveness as the project's prioritization target. The intent was that someone unfamiliar with the design could sit down at the board, navigate the menus, run any of the four modes, and understand the results without external instruction.

6.2.1 Design Decisions Made for the Prioritization Goal

Several decisions in the design exist specifically because of this goal. The UI is structured around six discrete screens rather than a single shared display, so each context (menu, entry, running, results) gets the full screen and a layout tuned to it. A USB mouse was added on top of the keypad and pushbuttons, so navigation feels familiar even though the rest of the system is custom hardware. Output that originally lived on the seven-segment display and the LEDs was moved into the on-screen UI, which removed the need to look away from the monitor to interpret status.

6.2.2 Results That Show Improvement Toward That Goal

The visible result is a system that holds together as a coherent product rather than a collection of working modules. The color palette is small and used consistently across every

screen, which makes interactive elements easy to recognize at a glance and keeps the look clean even with multiple values updating at once. The background is not static. A subtle animation runs behind the foreground content, which keeps the screen feeling alive while a mode is computing without distracting from the actual data. Each menu tile carries its own icon, and the corner logos give the UI a clear visual identity that ties the screens together. None of these elements are required by the rubric, but together they make the difference between an FPGA demo and something that looks like a finished tool.

The controls screen is reachable directly from the main menu and lays out every input device in one place. It explains which keypad keys are digits, which are backspace and clear, how to soft reset, and how to use the mouse to click between screens. A new user can read it once and have everything they need to operate the rest of the system. Navigation between screens is bidirectional and uses the same mouse-click model everywhere, with the on-board pushbuttons serving as a fallback. In informal testing, users were able to run all four modes after only a brief overview of the screen layout.

6.2.3 What Was Deferred Because of That Prioritization

A few features were considered but cut because they did not meaningfully improve the user experience for the time they would have cost. These include animated transitions between screens, a shooter minigame, and a richer color palette beyond the small set actually used. Mouse-driven scrolling through the full prime list was also considered but left out, since the existing "last few primes" display already covers the user-visible need without requiring a scrollable widget.

6.3 Final Functional Status by Requirement

This subsection tracks the rubric's primary functional requirements one by one and reports the final status of each on hardware.

Requirement 1a - Range Mode

Met. The system computes every prime between the user's entered upper/lower bounds, displays the total count and the elapsed time, and stores all primes in DDR2 for use in Test Mode.

Requirement 1b - Time Mode

Met. The system computes primes for the user's specified time in whole seconds and reports the total found at the end of the window. The full one-hour demo run executes correctly.

Requirement 2 - Single Mode

Met. The system accepts a single user-entered number, reports whether it is prime, and shows the elapsed time of the check.

Requirement 3 – Test Mode.

Met. The controller compares each prime stored in DDR2 against the matching prime parsed from the SD card in order, starting from 2. A clean run displays PASS in green along with the number of primes checked and stops on the capital "A" sentinel in the file. A mismatch stops immediately and displays FAIL in red along with both the DDR2 value and the SD card value. The "A" sentinel also handles the case where the SD card holds fewer primes than DDR2.

Requirement 4 – Accuracy and Prioritization.

Met. Range Mode and Single Mode results have been spot-checked against external prime tables, and Test Mode adds a continuous, large-scale check on every run. The chosen prioritization target was user experience, which is discussed in 6.2.

Requirement 5a – Display While Running.

Met. The generating screen shows the running prime count, the most recent primes found, the configured maximum value or maximum time depending on mode, and a continuously updated elapsed time.

Requirement 5b – Single Mode Display.

Met. The single-value screen shows the entered number, the prime or not prime result, and the elapsed time.

Requirement 5c – Test Mode Display.

Met. The test screen shows the running comparison count and the final PASS or FAIL outcome, with mismatched values shown on failure.

Requirement 5d – Sprite Use.

Met. A moving cursor sprite is overlaid on every screen, and additional sprite assets appear as corner logos and as menu-tile icons. The background pattern is constantly wrapping vertically, thus fulfilling the motion requirement as well.

Requirement 5e – Color Use.

Met. A small, consistent color palette is used across every screen to distinguish interactive elements, status text, and PASS or FAIL outcomes.

Requirement 6 – End-of-Mode Behavior.

Met. At the end of every mode, the displayed values are frozen and a visible "complete" indicator is shown. The user

must explicitly press "Back To Menu" or soft reset to leave the results.

6.4 Latency and Throughput

To characterize the throughput of the prime engine on hardware, the system was run in Time Mode for ninety minutes, and the largest prime found so far was recorded every fifteen minutes. The result is shown in Figure 4.

The curve rises sharply during the first interval, and the slope decreases over each later interval. This shape is the expected result of two effects that grow as the candidates get larger. First, the prime check runs trial divisors up to the square root of the candidate, and the subtraction-based modulus inside the check takes more cycles for larger numbers. Each individual check therefore costs more wall-clock time as the engine moves further out. Second, primes naturally thin out at higher values, so the engine has to walk through more non-prime candidates between each prime that actually gets recorded.

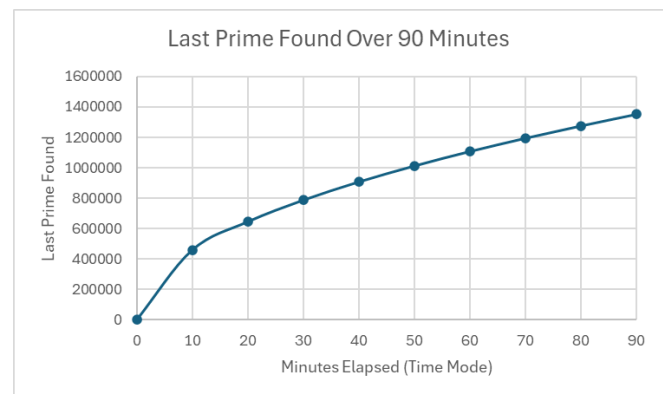


Figure 3: Graph of Recorded Last Primes over 90 Minutes

Together these two effects produce a curve that is steep early and gradually rolls off, which is exactly what the measurement shows. The fact that the measured data tracks the predicted shape confirms that the engine is running at full throughput and that no extra delay is being introduced by the framebuffer writes, the DDR2 storage path, or the UI's live updates while the mode is active.

The final metrics during both the 90 minute and primes between 0 – 1,000,000 can be seen below:

- **Primes Found in 90 Minutes:** 103,739
- **Time to find primes between 0 – 1,000,000:** 48 Minutes and 40 Seconds



Figure 4: Results Screen of 90 Minute Test

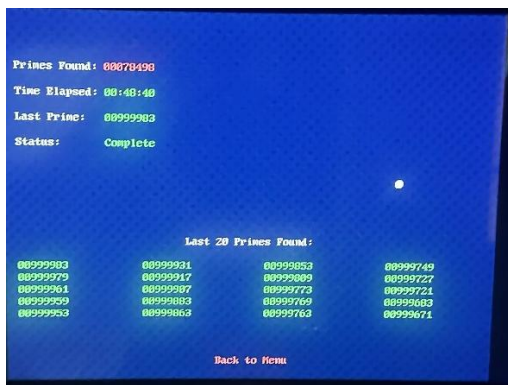


Figure 5: Results Screen of 1-1,000,000 Test.

6.5 Resource Utilization

Resource	Utilization	Available	Utilization %
LUT	9644	63400	15.21
LUTRAM	397	19000	2.09
FF	9657	126800	7.62
BRAM	8	135	5.93
DSP	16	240	6.67
IO	113	210	53.81
MMCM	2	6	33.33
PLL	1	6	16.67

Figure 6: Resource Utilization Table

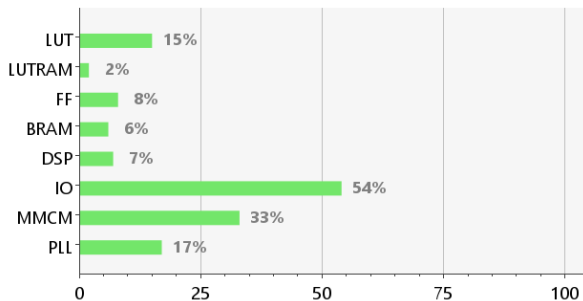


Figure 7: Resource Utilization Graph

6.6 Timing Performance

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -4.905 ns	Worst Hold Slack (WHS): 0.019 ns	Worst Pulse Width Slack (WPWS): 0.264 ns
Total Negative Slack (TNS): -127.184 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 108	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 25617	Total Number of Endpoints: 25615	Total Number of Endpoints: 10591

Timing constraints are not met.

Figure 8: Timing Performance

6.7 Tradeoffs and Limitations

The system design reflects a deliberate tradeoff between performance and usability, with priority given to a clean, interactive VGA-based user interface and overall system reliability. Resource utilization remains moderate, with LUT usage at approximately 15% and BRAM usage under 6%, indicating an efficient implementation with room for expansion. However, higher utilization is observed in I/O ($\approx 54\%$) and DSP resources ($\approx 53\%$), which is expected given the integration of multiple subsystems including VGA output, DDR2 memory, and input interfaces.

One of the primary limitations of the system is timing performance. The design does not fully meet timing constraints, with a worst negative slack (WNS) of approximately -4.9 ns and multiple failing endpoints. This is largely due to the complexity of integrating DDR2 memory, VGA rendering, and control logic across multiple clock domains. While the system functions correctly in practice, this indicates that further optimization, such as pipeline insertion, improved clock domain crossing strategies, or logic simplification, would be required for a fully timing-closed design.

Additionally, the use of a simple trial division algorithm for prime computation represents a tradeoff favoring design simplicity over computational efficiency. While sufficient for demonstration and functional correctness, more advanced algorithms could significantly improve performance for larger inputs.

Overall, the design prioritizes usability, modularity, and correctness over maximum performance, resulting in a robust and well-structured system with clear opportunities for future optimization.

VII. Reflections and Future Work

7.1 What Worked Well

Overall, the project reached a high level of completeness and reliability, with all major subsystems functioning as intended. Extensive testing and repeated validation provided strong confidence in the final system.

One of the most successful aspects of the design was the VGA user interface. The UI achieved a clean, professional

appearance while maintaining an intuitive and interactive user experience. Features such as button highlighting, mouse-based navigation, and keypad enhancements (including backspace and clear functionality) contributed to a polished and user-friendly system.

The modular, hierarchical design approach also proved highly effective. Breaking the system into smaller subsystems significantly simplified debugging and integration. Compared to previous projects with larger monolithic modules, this approach reduced debugging time and made it easier to isolate and fix issues.

Additionally, the prime computation subsystem integrated smoothly with minimal redesign. Once implemented, it required very few modifications, demonstrating strong initial design correctness.

Finally, the verification strategy was highly effective. Self-checking testbenches allowed rapid validation without relying on long synthesis cycles, which often exceeded 10 minutes. The inclusion of automated PASS/FAIL reporting and forced-failure cases ensured that the testbenches were both robust and trustworthy.

7.2 What Was Harder Than Expected

The most challenging aspect of the project was the DDR2 memory subsystem. Efficiently storing large amounts of prime data while maintaining manageable synthesis and integration complexity required careful architectural decisions.

The Test Mode implementation was also particularly difficult. Managing the interaction between SD card data and DDR2 storage introduced significant complexity, especially when handling large input files. Debugging this system required tracking data flow across multiple modules, which made isolating issues more challenging. This was ultimately resolved by redesigning the SD data handling approach to avoid loading the entire file at once, instead processing the data as a stream.

Integration between the SD parser and Test Mode controller also presented challenges, as synchronization and data flow needed to be carefully coordinated.

While the overall system complexity was high, the project was slightly more manageable than expected due to the reuse of a previously developed project foundation. This allowed more focus on integration and refinement rather than building every component from scratch.

Hardware testing largely aligned with simulation results, with only minor visual issues that were resolved through small adjustments.

7.3 What Would Be Done Differently Next Time

If the project were to be completed again, the primary change would be to start with DDR2-based storage from the beginning. Initially implementing storage using BRAM and later transitioning to DDR2 introduced unnecessary rework and slowed overall progress.

In terms of testing, the current approach proved effective and would largely be retained. The use of modular, self-checking testbenches significantly improved development efficiency compared to previous projects.

From a design perspective, most modules reached a strong final form and would not require major redesign. However, starting with the final intended architecture earlier would reduce integration overhead.

Given additional time, the focus would not be on simplifying the system, but rather on expanding it further with additional features while maintaining the same level of correctness and reliability.

7.4 Future Extensions

With additional time, several enhancements could be made to extend the functionality of the system.

One major extension would be the addition of a game mode, providing a more interactive and engaging user experience beyond the core project requirements. This would build upon the existing VGA and input infrastructure.

Performance improvements could also be achieved by implementing a faster prime computation algorithm, replacing the current trial division approach with a more efficient method.

From a user experience perspective, additional UI enhancements and animations could be introduced. While the current system already includes dynamic elements such as a moving background, further visual improvements could make the interface feel more responsive and engaging.

Overall, future work would focus on expanding functionality and improving performance while maintaining the strong design foundation established in this project.

7.5 Conclusion

This project successfully met all primary objectives, including prime computation, Test Mode validation, and the development of a fully interactive VGA-based user interface. The system demonstrates strong integration across multiple subsystems, including memory, input handling, and display logic.

The project's prioritization goal of ease of use and user interface quality was successfully achieved. The final system provides a smooth, intuitive, and visually polished experience that enhances usability beyond basic functional requirements.

While the development process presented several challenges, particularly in memory management and system integration, the final result reflects a high level of effort, refinement, and technical understanding.

Ultimately, this project represents a complete and well-executed system, and it is especially rewarding to see all components come together into a functional and showcase-ready design.

VIII. Team Contributions

8.1 Dawson's Contributions

During the development of this project, Dawson was primarily responsible for integrating the following:

- Prime Engine
- Mouse / Keypad Input + Cursor Display
- SD Card Communication + Test Mode
- Relevant Test Benches

8.2 Michael's Contributions

During the development of this project, Michael was primarily responsible for integrating the following:

- UI Layout & Design
- Keypad & Cursor Interactivity
- System Navigation
- Relevant Test Benches

8.3 Shared Work

Both members took turns diligently working during the MEMORY & VGA project, which laid the groundwork for VGA Output, Memory Management, Frame Buffering, and SD-Card Interaction in Optimus Prime. Documentation was also classified as shared work for this project, including the proposal, design presentation, check-in report, and final report.

References

1. <https://digilent.com/reference/programmable-logic/nexys-a7/start?srsId=AfmBOorZ0TRKzIzqMHMbhkn2GiIxmqlX2UudW6qzvc7tTinHJc2ER6>
2. <https://github.com/Digilent/digilent-xdc/blob/master/Nexys-A7-100T-Master.xdc>
3. <https://github.com/Herring-UGACSEE-4280-S26/blake-4280>
4. https://github.com/Herring-UGACSEE-4280-S26/tools/blob/main/nexys4ddr_mig_prj.zip

Appendix A. Vivado IP Walkthroughs

This appendix documents the Vivado IP blocks used in the project. For each block it lists what the block does, why it was chosen over hand-written RTL, the steps used to instantiate and customize it in Vivado, how it is connected into the rest of the design, and how it was tested.

The project uses two IP blocks from the Vivado IP catalog: a PLL generated through the Clocking Wizard, and a DDR2 memory controller generated through the MIG 7-Series IP. A handful of smaller memories (the font ROM, the sprite ROMs, the SD prime cache, and the cross-domain FIFOs) are written in plain Verilog and inferred as Block RAM by the synthesis tool. Those are listed at the end of this appendix for completeness.

A.1 PLL - Xilinx Clocking Wizard

What it does: The PLL takes the 100 MHz clock that comes from the board's onboard oscillator and produces four output clocks at the rates the design needs: 200 MHz for the DDR2 reference clock, 100 MHz for the main system logic, 50 MHz for the SD controller, and 25 MHz for the VGA pixel clock. It also produces a locked signal that goes high once all four output clocks are stable.

Why we used it: Generating these clocks in user logic would require a chain of counters and would not give clean, phase-aligned outputs. The Clocking Wizard wraps a hardware MMCM/PLL primitive that already exists on the FPGA fabric. Using the IP wizard avoids hand-instantiating a vendor primitive and lets us change the output frequencies later just by re-running the wizard.

Steps to instantiate in Vivado:

1. In the Flow Navigator, click IP Catalog.
2. Under FPGA Features and Design → Clocking, double-click Clocking Wizard.
3. On the Clocking Options tab, leave the primitive set to MMCM and the input source set to a single-ended clock. Set the input frequency to 100.000 MHz.
4. On the Output Clocks tab, enable four output clocks. Set them to 200, 100, 50, and 25 MHz. Rename them to `clk_mem`, `clk_cpu`, `clk_sd`, and `clk_vga`. Enable the locked port. Disable any reset ports that are not needed and rename the active-low reset to `resetn`.
5. On the Port Renaming tab, confirm the input is named `clk_in` to match the project's top-level wiring.
6. Click OK, then Generate, and let Vivado synthesize the IP out of context.

How it is interfaced: The PLL is instantiated once at the top level inside `top.v` as `u_pll`. Its `clk_in` is driven from the `clk100mhz` board pin and its `resetn` is driven from the board's CPU RESET button. Its four output clocks fan out

to every clocked module in the design. Its locked output is ANDed with the external reset to form the project-wide synchronous reset `sys_resetn_w`, so that no logic comes out of reset until all four clocks are running.

How it was tested: During bring-up, the locked signal was tied to one of the user LEDs to confirm that the PLL came out of calibration shortly after power-up. Each output clock was confirmed by progressively bringing up modules that depended on it: the VGA path confirmed `clk_vga`, the SD reader confirmed `clk_sd`, the MIG calibration handshake confirmed `clk_mem`, and the rest of the design running confirmed `clk_cpu`.

A.2 DDR2 Memory Controller - MIG 7-Series

What it does: MIG drives the board's external DDR2 SDRAM chip. It handles the initialization and calibration of the memory, the timing of read and write commands, and the conversion between the design's wide user-side data path and the narrower physical bus that goes to the chip. From the design's point of view, MIG presents a simple command port and a wide data port.

Why we used it: Writing a DDR2 controller from scratch is well outside the scope of this project. MIG is the vendor-supplied controller for this exact memory chip on this exact board, and Digilent provides the timing parameters needed to configure it. Using MIG also gives us a clean user-side interface that hides all of the DDR2-specific timing details.

Steps to instantiate in Vivado:

1. In the Flow Navigator, click IP Catalog.
2. Under Memories & Storage Elements → Memory Interface Generators, double-click Memory Interface Generator (MIG 7 Series).
3. Step through the wizard. On the device pins page, select Verify Pin Changes and load the Nexys A7 DDR2 pin file provided by Digilent (or enter the pin assignments by hand from the board master constraints).
4. On the controller options page, set the memory part to MT47H64M16HR-25E, the data width to 16 bits, and the input reference clock to 200 MHz.
5. On the user interface page, confirm the user-side data width as 128 bits and accept the default ordering.
6. On the system clock page, set the system clock source to "No Buffer" and connect the design's `clk_mem` output directly into MIG.
7. Click Generate and let Vivado synthesize the IP out of context.

How it is interfaced: MIG is instantiated at the top level. Its physical-side ports go straight out to the DDR2 pins on the FPGA. Its user-side ports go into two small bridge modules:

ddr2_fb_cdc for the framebuffer traffic and prime_ddr_bridge for the prime-list traffic. Both bridges contain async FIFOs because MIG runs on its own ui_clk and the rest of the design runs on clk_cpu. The init_calib_complete flag from MIG is brought across into clk_cpu and gates the rest of the design so that no module tries to read or write DDR2 until calibration is finished.

How it was tested: During bring-up, init_calib_complete was wired to a user LED so we could confirm calibration finished after power-up. After that, the framebuffer path was tested by writing a known color pattern into DDR2 and watching it come out on the VGA monitor. The prime-list path was tested by writing a small known sequence of values, reading them back, and comparing them against the SD-card source through the test-mode comparator.

A.3 Inferred Memories

The remaining memories in the design are written as plain Verilog arrays and inferred as Block RAM by the synthesis tool. They are not generated through the IP catalog and therefore do not have wizard screenshots. They are listed here for completeness:

- **font_rom_ascii** - the 8×16 font glyph table used by every text-rendering screen.
- **Cursor sprite ROM and corner logo bitmaps** - small 32×32 1-bit-per-pixel bitmaps stored as packed constants in Verilog.
- **sd_prime_cache** - the cache that holds primes parsed from the SD card at boot for replay during test mode.
- **SD-byte feeder FIFO and prime-queue FIFO** - small queues used to buffer data crossing between the SD, CPU, and MIG clock domains.

Appendix B. Simulation Results

B.1 Prime Check Core

```
PASS : randomized case candidate=4747 result=0
PASS : randomized case candidate=1582 result=0
PASS : randomized case candidate=1445 result=0
PASS : randomized case candidate=3688 result=0
PASS : randomized case candidate=567 result=0
PASS : randomized case candidate=3938 result=0
PASS : randomized case candidate=2657 result=1
PASS : randomized case candidate=612 result=0
PASS : randomized case candidate=2443 result=0
PASS : randomized case candidate=4630 result=0
PASS : randomized case candidate=4701 result=0
PASS : randomized case candidate=1984 result=0
PASS : randomized case candidate=103 result=1
PASS : randomized case candidate=1346 result=0
PASS : t returned DUT to idle correctly
PASS : t returned DUT to idle correctly
```

```
tb_prime_check_core complete
Total tests run : 324
Total passes   : 324
Total errors   : 0
RESULT        : PASS
```

Figure B1: Terminal Output of Prime Check Core

B.2 SD Parser

```
PASS : randomized stream line value=76334
PASS : randomized stream line value=89333
PASS : randomized stream line value=22096
PASS : randomized stream line value=18287
PASS : randomized stream line value=33602
PASS : randomized stream line value=17977
PASS : randomized stream line value=39716
PASS : randomized stream line value=52691
PASS : randomized stream line value=83958
PASS : randomized stream line value=89789
PASS : randomized stream line value=88536
PASS : op marker : stream_done asserted
```

```
tb_sd_prime_parser complete
Total tests run : 70
Total passes   : 70
Total errors   : 0
RESULT        : PASS
```

Figure B2: Terminal Output of SD Parser

B.3 SD Feeder

```
PASS : randomized final drain value=62622
PASS : randomized final drain value=65187
PASS : randomized final drain value=37497
PASS : randomized final drain value=23032
PASS : randomized final drain value=98583
PASS : randomized final drain value=2284
PASS : randomized final drain value=62302
PASS : randomized final drain value=81446
PASS : randomized final drain value=36668
PASS : randomized final drain value=25954
PASS : al eof : sd_end_of_file asserted
```

```
tb_sd_prime_feeder complete
Total tests run : 59
Total passes   : 59
Total errors   : 0
RESULT        : PASS
```

Figure B3: Terminal Output of SD Feeder

B.4 Input Controller

```

PASS : D : key_valid low after release
PASS : key D : no key_press on release
PASS : keypad key E : key_press
PASS : keypad key E : key_valid high
PASS : keypad key E : key_code correct
PASS : : no repeat key_press while held
PASS : E : key_valid low after release
PASS : key E : no key_press on release
PASS : keypad key F : key_press
PASS : keypad key F : key_valid high
PASS : keypad key F : key_code correct
PASS : : no repeat key_press while held
PASS : F : key_valid low after release
PASS : key F : no key_press on release

-----
tb_input_ctrl complete
Total tests run : 129
Total passes   : 134
Total errors   : 0
RESULT        : PASS

```

Figure B4: Terminal Output of Input Controller

B.5 Mouse Subsystem

```

PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet
PASS : randomized packet

-----
tb_mouse_subsystem complete
Total tests run : 84
Total passes   : 84
Total errors   : 0
RESULT        : PASS

```

Figure B5: Terminal Output of Mouse Subsystem

B.6 VGA Output with Cursor

```

PASS : center cursor with timing checks
PASS : sweep : top-left clipped cursor
PASS : ep : bottom-right clipped cursor
PASS : eep : randomized cursor position
PASS : eep : randomized cursor position
PASS : eep : randomized cursor position

-----
tb_vga_output_with_cursor complete
Total tests run : 7
Total passes   : 7
Total errors   : 0
RESULT        : PASS

```

Figure B6: Terminal Output of VGA output with cursor

B.7 Prime Controller

```

PASS : domized range case : event count
PASS : domized range case : event value
PASS : domized range case : event value
PASS : domized range case : event value
PASS : domized range case : event value
PASS : range case : recent_valid_count
PASS : ange case : recent history entry
PASS : ange case : recent history entry
PASS : ange case : recent history entry
PASS : ange case : recent history entry
PASS : case : single_is_prime inactive

-----
tb_prime_controller complete
Total tests run : 463
Total passes   : 463
Total errors   : 0
RESULT        : PASS

```

Figure B7: Terminal Output of Prime Controller

B.8 DDR Prime Storage

```

PASS : randomized readback addr=63 value=0
PASS : randomized readback addr=37 value=0
PASS : randomized readback addr=11 value=0
PASS : randomized readback addr=47 value=0
PASS : randomized readback addr=1 value=29
PASS : randomized readback addr=48 value=0
PASS : randomized readback addr=0 value=23
PASS : randomized readback addr=3 value=0
PASS : randomized readback addr=22 value=89791
PASS : randomized readback addr=62 value=65159
PASS : randomized readback addr=48 value=0
PASS : randomized readback addr=51 value=29240
PASS : randomized readback addr=41 value=0
PASS : ffc stored_count : stored_count

-----
tb_prime_ddr_storage complete
Total tests run : 34
Total passes   : 34
Total errors   : 0
RESULT        : PASS

```

Figure B8: Terminal Output of DDR Prime Storage

B.9 Test Mode Controller

```

PASS : mismatch case : fail_stored_val
PASS : ndom mismatch case : fail_sd_val
PASS : mismatch case : test_running low
PASS : ndom mismatch case : test_passed
PASS : ndom mismatch case : test_failed
PASS : m mismatch case : no_data_stored
PASS : m mismatch case : primes_checked
PASS : mismatch case : fail_stored_val
PASS : ndom mismatch case : fail_sd_val

-----
tb_test_mode_ctrl complete
Total tests run : 142
Total passes   : 142
Total errors   : 0
RESULT        : PASS

```

Figure B9: Terminal Output of Test Mode Controller

B.10 Prime Subsystem

```

PASS : ond : prime_count matches events
PASS : nd : stored_count matches events
PASS : time mode one second storage addr=0 value=2
PASS : time mode one second storage addr=1 value=3
PASS : time mode one second storage addr=2 value=5
PASS : time mode one second storage addr=3 value=7
PASS : time mode one second storage addr=4 value=11
PASS : time mode one second storage addr=5 value=13
PASS : time mode one second storage addr=6 value=17
-----
tb_prime_subsystem complete
Total tests run : 400
Total passes   : 400
Total errors   : 0
RESULT        : PASS
-----

```

Figure B10: Terminal Output of Prime Subsystem

B.11 Top-level Smoke Test

```

PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
PASS : random navigation keeps display_mode legal and non-X
-----
tb_top_smoke complete
Total checks : 40
Total errors : 0
-----

```

Figure B11: Terminal Output of Top level smoke test